

**Hackers vs IA:**

(1. Práctica) Una organización clandestina de hackers realiza misiones contra grandes corporaciones tecnológicas. El sistema debe modelar distintos tipos de agentes capaces de ejecutar misiones digitales.

Clase Misión Cada misión tiene un nombre y un nivel de dificultad.

Clase abstracta Agente Todo agente posee: alias, energía y nivel_hackeo.

- La energía inicial no puede ser negativa (lanzar ValueError si se intenta).
- El nivel de hackeo debe estar entre 1 y 10 inclusive (lanzar ValueError si no).

Todo agente debe implementar:

- `ejecutar_mision(mision)` → retorna True si la misión fue exitosa, False en caso contrario. Una misión es exitosa si el agente tiene energía suficiente y su `poder_hackeo()` es mayor o igual a la dificultad de la misión.
- `poder_hackeo()` → retorna un entero según el tipo de agente.

Todo agente puede usar:

- `recargar_energia(cantidad)` → incrementa su energía en la cantidad indicada.

Tipos de agentes

- **Cracker:** Consume 30 puntos de energía al ejecutar una misión. Si no tiene energía suficiente, no puede ejecutarla. Su poder de hackeo es: `nivel_hackeo * 2`.
- **IngenieroSocial:** Consume 10 puntos de energía al ejecutar una misión. Si no tiene energía suficiente, no puede ejecutarla. Su poder de hackeo es: `nivel_hackeo + 5`.
- **IAAutonoma:** Necesita al menos 15 puntos de energía para ejecutar una misión. Si está por debajo, **recarga**

automáticamente la energía que le falte para llegar a 15 antes de intentar ejecutarla. Su poder de hackeo es: `nivel_hackeo * 3`.

Clase Operativo

Implementar el método `ejecutar_operativo(agentes, misiones)` que reciba una lista de agentes y una lista de misiones, y haga que cada agente ejecute la misión en su misma posición (el agente 0 ejecuta la misión 0, el agente 1 la misión 1, etc.). Al finalizar, el método debe:

- Informar cuántas misiones fueron exitosas.
- Listar los alias de los agentes que las cumplieron.

(2. Teoría) En el método `ejecutar_operativo(agentes, misiones)` no se permite verificar el tipo concreto de cada objeto.

1 - ¿Qué concepto de POO se está aplicando?

2 - ¿Por qué utilizar `if type(...)` o `isinstance(...)` suele considerarse una mala práctica en este contexto?

3 - ¿Qué ventajas aporta esta solución al diseño del sistema?

4 - Analizar el caso particular de `IAAutonoma`: su comportamiento en `ejecutar_mision()` es diferente al de los otros agentes porque recarga energía automáticamente antes de intentar la misión. ¿Eso rompe o respeta el contrato definido por la clase base? Justificá tu respuesta.

Requisitos generales: Utilizar encapsulamiento correctamente. Se espera utilizar propiedades cuando resulte necesario para mantener las restricciones del sistema. Evitar acceso directo innecesario a atributos. Utilizar herencia cuando corresponda. Reutilizar código evitando duplicación. Mantener una buena organización y legibilidad del código. Indentar correctamente el código.

**Hackers vs IA:**

(1. Práctica) Una organización clandestina de hackers realiza misiones contra grandes corporaciones tecnológicas. El sistema debe modelar distintos tipos de agentes capaces de ejecutar misiones digitales.

Clase Misión Cada misión tiene un nombre y un nivel de dificultad.

Clase abstracta Agente Todo agente posee: alias, energía y nivel_hackeo.

- La energía inicial no puede ser negativa (lanzar ValueError si se intenta).
- El nivel de hackeo debe estar entre 1 y 10 inclusive (lanzar ValueError si no).

Todo agente debe implementar:

- `ejecutar_mision(mision)` → retorna True si la misión fue exitosa, False en caso contrario. Una misión es exitosa si el agente tiene energía suficiente y su `poder_hackeo()` es mayor o igual a la dificultad de la misión.
- `poder_hackeo()` → retorna un entero según el tipo de agente.

Todo agente puede usar:

- `recargar_energia(cantidad)` → incrementa su energía en la cantidad indicada.

Tipos de agentes

- **Cracker:** Consume 30 puntos de energía al ejecutar una misión. Si no tiene energía suficiente, no puede ejecutarla. Su poder de hackeo es: `nivel_hackeo * 2`.
- **IngenieroSocial:** Consume 10 puntos de energía al ejecutar una misión. Si no tiene energía suficiente, no puede ejecutarla. Su poder de hackeo es: `nivel_hackeo + 5`.
- **IAAutonoma:** Necesita al menos 15 puntos de energía para ejecutar una misión. Si está por debajo, **recarga**

automáticamente la energía que le falte para llegar a 15 antes de intentar ejecutarla. Su poder de hackeo es: `nivel_hackeo * 3`.

Clase Operativo

Implementar el método `ejecutar_operativo(agentes, misiones)` que reciba una lista de agentes y una lista de misiones, y haga que cada agente ejecute la misión en su misma posición (el agente 0 ejecuta la misión 0, el agente 1 la misión 1, etc.). Al finalizar, el método debe:

- Informar cuántas misiones fueron exitosas.
- Listar los alias de los agentes que las cumplieron.

(2. Teoría) En el método `ejecutar_operativo(agentes, misiones)` no se permite verificar el tipo concreto de cada objeto.

1 - ¿Qué concepto de POO se está aplicando?

2 - ¿Por qué utilizar `if type(...)` o `isinstance(...)` suele considerarse una mala práctica en este contexto?

3 - ¿Qué ventajas aporta esta solución al diseño del sistema?

4 - Analizar el caso particular de `IAAutonoma`: su comportamiento en `ejecutar_mision()` es diferente al de los otros agentes porque recarga energía automáticamente antes de intentar la misión. ¿Eso rompe o respeta el contrato definido por la clase base? Justificá tu respuesta.

Requisitos generales: Utilizar encapsulamiento correctamente. Se espera utilizar propiedades cuando resulte necesario para mantener las restricciones del sistema. Evitar acceso directo innecesario a atributos. Utilizar herencia cuando corresponda. Reutilizar código evitando duplicación. Mantener una buena organización y legibilidad del código. Indentar correctamente el código.